

Документация на проекта

Използвани средства и структура на проекта

За разработката на проекта са използвани: CLion като IDE, CMake като build система, Doctest^[1] за тестване на приложението и Git за version control. Проектът има и хранилище^[2] в GitHub.

Структура на проекта

- src/ - съдържа source файловете (.cpp)
- include/ - съдържа всички хедър файлове (.h)
- tests/ - съдържа няколко тестови файлове за програмата (.h)
- doc/ - тук се съдържа документацията
- third_party/ - тук се съдържа чужд код, конкретно doctest.h библиотеката.

Подход, алгоритми и проблеми

Архитектура

Приложенията с потребителски интерфейс много често се разделят на 2 логически цялости - “Data” слой и “View” слой. В проекта се използва поведенчески шаблон от Command-и за комуникирането м/у тях.

Съществува централен клас Application, който композира всички ключови елементи на приложението - “running” флаг, таблица, layout на таблицата и др.

Команди

За всички взаимодействия м/у потребителя и приложението, трябва да има съответната команда. От изискванията в заданието, съществуват командите: SaveCommand, SaveAsCommand, OpenCommand, PrintCommand и EditCellCommand, а някои допълнителни “Quality of Life” команди са HelpCommand и QuitCommand.

Всяка команда работи в рамките на някакъв контекст, затова повечето класове на команди съдържат или `Application& app` поле, или `Table& table` поле.

Потенциален проблем в `PrintCommand` е “скъпото” обхождане на таблицата, за да се изчислят широчините на редовете/колоните, за да може таблицата да изглежда подравнена при извеждане на екрана. Това може да се разреши с клас, който следи `layout`-а на таблицата (в проекта това е `TableLayout`). За реализирането на това следене на широчините се използва `Observer` шаблон, което позволява `layout`-ът да се изчислява само тогава, когато таблицата се редактирала на дадена позиция.

За `parse`-ването на команди се използва дърво като структура от данни, като всеки `Node` представлява възможна дума, с която `parser`-ът може да `match`-не. Например за “`save`” и “`save as`” командите, “`save`” ще е собствен `Node` и ще има дете, което е само думата “`as`”. Допълнително всеки `Node` трябва да има `CommandAction` - callback функция, построяваща съответните командните обекти (например `SaveAsCommand`), която се извиква при успешно `match`-ване.

Дотук `parse`-рът може да работи само с команди без аргументи. За да може да работи с аргументи, запазваме ключова дума за тях, в програмата това е “`<arg>`”. Сега като `parse`-ваме низ, ако срещнем `Node`, който представлява аргумент, тогава продължаваме надолу по дървото, като накрая подаваме аргумента на `CommandAction`-а.

Последния проблем е, че съществуват команди, които като последен аргумент приемат низ с интервали, например `edit 1 1 “hello, world!”` или командите за четене или записване във файл (тогава името на файл-а може да съдържа интервали). За този проблем въвеждаме 2-ра ключова дума - “`...`”, която инструктира `parser`-ът да “събере” всички останали аргументи в един низ, когато е `match`-нал всички възможни аргументи от тип “`<arg>`”.

Пример:

Създаване на команда от конструктора на Application:

```
parser.registerCommand("edit <arg> <arg> ...",
    [](const std::vector<std::string>& args, Application *ctx) {
        // създаваме обект от тип EditCellCommand
    },
    this
);
```

Match-ване на командата:

```
EditCellCommand* cmd = parser.parse("edit 10 10 \"cell text\"");
cmd->execute();
```

Клетки

Проблемът с различните типове на клетка е решен с runtime полиморфизъм, вместо например tagged union-и, понеже чрез виртуални методи, различните видове клетки могат да имат различни поведения много лесно, докато tagged union-и обикновено изискват switch-case оператори навсякъде, където искаме различно поведение.

Налагат се 2 различни метода за преобразуването на клетка към низ, понеже при принтиране искаме да видим “ефективните” стойности на клетките (например за формула, това е изчислената стойност). Обаче при записване във файл, ако просто запишем стойността на формулата, при прочитане губим информация. Затова в абстрактния базов клас Cell, съществуват и двата метода. (asString() е за принтиране, а serialize() е за записване във файл)

За инстанцирането на различните видове клетки е уместно да се използва шаблона Simple Factory. В проекта това е CellFactory, като той съдържа метод за създаване на клетка при подаден тип, и метод, който създава клетка от правилния тип автоматично.

Формули

Формулите се реализират с Abstract Syntax Tree (AST) структура от данни, като parse-ването на формула-низ се случва рекурсивно и използва Shunting yard algorithm (Dijkstra) за бинарните операции.

Във формула всички типове се превръщат в десетично число (double), затова в проекта няма нужда от система за различни типове във формулите и правила за преобразуването м/у тях.

Възможно е една формула да е “скъпа” за изчисление, затова в класа за клетка-формула има поле, което кешира изчислената стойност, като кешът се инвалидира, когато в таблицата се направи запис на клетка. Това означава, че при поредни принтирания на таблицата, стойностите на формулите са точно кешираните стойности.

Може да си представим формулите и зависимостите м/у тях като насочен ацикличен граф (DAG). Графът трябва да е ацикличен, понеже при цикъл оценяването на формулите става невъзможно. Частен случай е когато дадена формула сочи към собствената си клетка - тогава това представлява примка в графа.

Трябва да се забранява добавянето на формула, която ще създаде цикъл, за тази цел в класа за формулите може да се добави флаг `isEvaluating_`, който се включва, когато почнем рекурсивно да оценяваме децата на текущата формула и се изключва, когато приключим. Така в началото на функцията, ако `isEvaluating_` е истина, тогава това означава, че вече сме посетили формулата, т.е. имаме цикъл.

Таблица

Таблицата може да се представи, като матрица с максимален брой редове и колони. Искаме, таблицата да може да расте динамично, за да заделяме пропорционално количество памет на тази която всъщност използваме.

Както бе споменато по-нагоре, трябва ни Observer шаблон, чрез който да следим широчината на редовете/колониите. За тази цел, има абстрактен базов клас `TableObserver`, който декларира чисто виртуален метод `onCellChanged()`. Сега `TableLayout` може да наследи този клас и да изчисли новата широчина на реда/колоната, в която току що сме променили клетка. За да информира observer-ите, класът на таблицата трябва да има 2 метода, с които да се добавят и премахват observer-и: `subscribe()` и `unsubscribe()`,

както и метод, който извиква `onCellChanged()` (това е `Table::notifyOnCellChanged()` в програмата).

Класове

Класове свързани с командите

- `class Command:`
 - Представава абстрактен базов клас на команда
 - Методи:
 - `virtual void execute() = 0`
- Видове команди:
 - `class SaveCommand (Application&)`
 - `class SaveAsCommand (Application&, std::string path)`
 - `class OpenCommand (Application&, std::string path)`
 - `class HelpCommand (Application&)`
 - `class QuitCommand (Application&)`
 - `class PrintCommand (Application&)`
 - `class ClearCommand (Table *)`
 - `class EditCellCommand (Table*, size_t, size_t, std::string)`
- `class CommandParser:`
 - `struct CommandAction:`
 - Ламбдите, които не улавят променливи декауват към обикновени указатели към функции - като тези от C. Използвам тази структура да опаковам простия callback, който е ламбдата заедно с контекста, който се подава в него (това е алтернативен подход, понеже не може да се ползва `<functional>` за този проект)
 - Полета:
 - `CommandCallback cb`
 - `Application* ctx`

`CommandCallback` е синоним на указател на функция от следния тип:

```
using ... = Command*(*)(const std::vector<std::string>&, Application *);
```

- Метод:
 - `Command* operator()(const std::vector<std::string> &)`
`const`
- `class Node:`
 - Полета:
 - `std::string lexeme`
 - `CommandAction action;`
 - `std::vector<Node*> children`
 - Методи:
 - `bool isTerminal() const`
 - `bool isLeaf() const`
 - `Node* findChild(std::string& lexeme)`
 - `Node* addChild(const Node&)`
 - `bool removeChild(const Node&)`
- Поле:
 - `Node root`
- Методи:
 - `void registerCommand(std::string&, CommandCallback& cb, Application *ctx)`
 - `Command *parse(std::string&)`

Класове свързани с клетките

- `class Cell:`
 - Представява абстрактен базов клас на клетка
 - Методи:
 - `virtual Cell* clone()`
 - `virtual double asValue()`
 - `virtual std::string asString()`
 - `virtual std::string serialize()`
- `class IntegerCell : public Cell:`
 - Поле: `int32_t value`
- `class DecimalCell: public Cell:`
 - Поле: `double value`

- class StringCell: public Cell
 - Поле: std::string text

- class DateCell: public Cell:
 - Полета:
 - std::chrono::year_month_day date

- class FormulaCell: public Cell
 - Представява клетка съдържаща формула
 - Полета:
 - std::string representation
 - FormulaASTNode *formula
 - size_t lastVersion
 - double cachedValue
 - bool isError_, isEvaluating_

- class CellFactory
 - Методи:
 - template<class CellType, class... Args>
static Cell* create_cell(Args&&... fields)
 - static Cell* create_cell_auto(const std::string& text)

Класове свързани с реализирането на формулите

- class FormulaASTNode
 - Абстрактен клас за описанието на синтаксиса на една формула.
 - Метод:
 - virtual double evaluate()

- class FormulaOperand : public FormulaASTNode
 - Абстрактен клас за операнд във формулата - във случая, това може да е “референция” към клетка или литерал.

- class FormulaCellReference: public FormulaOperand
 - Полета:
 - size_t row
 - size_t column
- class FormulaNumberLiteral: public FormulaOperand
 - Поле: double value
- class FormulaParser:
 - Полета:
 - std::string source
 - size_t position
 - Методи:
 - bool isValid() const
 - char peek() const
 - bool consume(char symbol)
 - bool consumeCaseInsensitive(char symbol)
 - bool consumeWhitespaces()
 - ...
 - FormulaCellReference* consumeCellReference()
 - FormulaNumberLiteral* consumeNumberLiteral()
 - FormulaASTNode* consumeExpression()
 - static FormulaASTNode* parseFormula(const std::string& text)

Класове за изключение при изчисления на клетки

- class cell_evaluation_error : public std::runtime_error
- class cell_evaluation_critical_error : public std::runtime_error
- class formula_division_by_zero_error :
 - public cell_evaluation_error
- class formula_cyclic_dependency_error :
 - public cell_evaluation_critical_error

Идеята зад тези изключения е да има разграничение м/у грешки, които са допустими в таблица, като например делението на нула и грешки, които изобщо не трябва да постъпват в таблицата, като например присъствието на цикъл при оценяването на формулите.

Класове свързани с таблицата

- class Table
 - Представява таблицата
 - Полета:
 - size_t rows, cols
 - size_t rowsCapacity, colsCapacity
 - std::vector<Cell *> cells
 - size_t globalVersion
 - std::vector<TableObserver *> listeners
 - Методи:
 - void reserve(size_t, size_t)
 - size_t getRows(), size_t getCols()
 - Cell* getCell(size_t row, size_t col)
 - void clear()
 - void setCell(size_t row, size_t col, Cell* cell)
 - std::vector<Cell*> getRow()
 - std::vector<Cell*> getCol()
 - void subscribe(TableObserver *listener)
 - void unsubscribe(const TableObserver *listener)
- class TableObserver
 - Абстрактен клас, който “слуша” за промени в таблицата, например при Table::setCell(...) се извиква onCellChange(...)
 - Метод:
 - virtual void onCellChanged(
 size_t row,
 size_t col,
 const Cell* cell
) = 0
 - virtual void onTableCleared() = 0
- class TableLayout : public TableObserver
 - Следи широчините на всеки ред и колона от таблицата (това е полезно при принтиране).

- Полета:
 - `std::vector<size_t> rowHeights`
 - `std::vector<size_t> colWidths`
- Методи:
 - `size_t getRowHeight(size_t row)`
 - `size_t getColWidth(size_t col)`
 - `void clear()`
 - `void updateFromCell(Table* table, size_t row, size_t col)`
 - `void updateFromTable(Table* table)`
 - `void onCellChanged(Table *table, size_t row, size_t col)`
override
 - `void onTableCleared()` override
- `class TableSerializer`
 - Методи:
 - `static Table* loadFromFile(std::string&, char)`
 - `static void saveToFile(Table*, std::string&, char)`

Бъдещи подобрения

Ако искаме да можем да използваме клетки на ред/колона по-голяма от, например 10000, тогава ще е необходимо да се задели 2D матрица с размери 10000x10000, което е много памет. По-добро решение е, в класа `Table`, да се запазват клетките в хештаблица, в която ключовете са позиции на клетки, а стойностите са обекти от тип `Cell`.

В текущите `Table` и `TableObserver`, е доста лесно да се добавят още методи, които да обогатят възможните взаимодействия с таблицата, например:

- `Table::deleteRow()` → `TableObserver::onDeleteRow()`
- `Table::deleteCol()` → `TableObserver::onDeleteCol()`

Може изчислението на формулите да се подобри, ако всяка формула следи от кои формули зависи, и кои формули зависят от нея. Тогава може точно да се прецени кои формули е нужно да се преизчислят и кои имат валидна кеширана стойност.

Ще е полезно, ако се имплементира възможност за синоними на команди, например, ако командата “quit” има alias-и, “exit” и “close”, тогава те да са функционално еднакви. Друг пример е вместо “edit”, да може да се напише “replace” или “set”.

Силно подобрение ще бъде употребата на умни указатели, като `std::unique_ptr` и `std::shared_ptr`, понеже те предоставят автоматично освобождаване на паметта и улесняват преместването на обекти.

Литература

1. “doctest/doctest: The fastest feature-rich C++11/14/17/20/23 single-header testing framework · GitHub.” *GitHub*, <https://github.com/doctest/doctest>.
2. “A CLI spreadsheet editor project for an OOP course.” *GitHub*, https://github.com/BvHand487/spreadsheet_editor.